SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Section / Batch:	AIML	**Date:**	

Code of Conduct

I P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.
However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input **Example customer statement:**

"I reported the damage to the car after the accident."
The sentence can produce different interpretations depending on whether **"to the car"** is associated with *reported* or *damage*.

Tasks:

a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.

b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.

c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.

d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

---

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.
The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries **Example passenger request:**

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."
The system sometimes incorrectly associates **"with AC sleeper"** with the passengers instead of the ticket/class information.

Tasks:

a) Analyze the possible syntactic interpretations of the given passenger request using CFG.

b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.

c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.

d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

---

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations
- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

a) Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.

b) Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.

c) Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.

d) Develop a step-by-step processing workflow showing how the system can extract:

- Supplier
- Action
- Equipment
- Company
- Obligation
- Deadline and justify how the proposed approach improves automated contract analysis.

---

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter naturallanguage queries rather than selecting predefined filters. The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements **Example user query:**

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate **"for students"** with the touchscreen rather than the laptop.

Tasks:

a) Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.

b) Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.

c) Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.

d) Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

---

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users **Example spoken command:**

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, **"with two passengers"** may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore." **Tasks:**

a) Analyze the possible syntactic structures and ambiguities in the given voice command.

b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.

c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.

d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1	4	3	3	4	4	3	20	88
2	4	4	4	3	3	3	18	
3	4	3	3	4	4	3	18	
4	4	3	3	4	4	4	16	
5	4	4	3	4	3	4	16	

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURABAN		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

a) Structural ambiguity using CFG

The sentence is:

"I reported the damage to the car after the accident."

The sentence can be represented using a simplified Context-Free Grammar (CFG):

S → NP VP

VP → V NP PP

NP → Det N

NP → Pronoun

PP → P NP

The basic structure is:

S

├── NP → I

└── VP

  ├── V → reported

  ├── NP → the damage

  └── PP → to the car

However, the prepositional phrase **"to the car"** can have different attachments.

Interpretation 1 – "to the car" modifies the damage

[I reported [the damage [to the car]] [after the accident]] Meaning:

The reported damage was damage **to the car**.

Interpretation 2 – "to the car" modifies the reporting action

[I reported [the damage] [to the car] [after the accident]]

Meaning: The speaker reported the damage **to the car/location/person associated with the car**, although this interpretation is less natural in normal insurance language.

The ambiguity occurs because CFG rules allow the PP to attach either to the noun phrase or the verb phrase.

b) Limitations of basic CFG

A basic CFG is useful for identifying grammatical structures, but it has several limitations in insurance claim processing.

1. Structural ambiguity

CFG can generate multiple valid parse trees for the same sentence. It does not inherently know which interpretation is more appropriate.

2. No probability

All grammatically valid structures are normally treated equally. The parser cannot determine which interpretation is most likely.

3. Agreement problems

Basic CFG may allow incorrect combinations such as:

The customer report the damage.

instead of:

The customer reports the damage.

CFG does not automatically enforce grammatical features such as number and person.

4. Long sentences

Insurance claims may contain several clauses, descriptions, dates, locations, and conditions. The number of possible parse structures can increase significantly.

5. Conversational input

Customers may provide incomplete statements such as:

Car damaged yesterday.

Need claim for 50,000.

Accident near Chennai.

A strict CFG expects a more complete grammatical structure and may fail to parse such input correctly.

6. Domain-specific language

Insurance terminology such as *policy number, deductible, claimant, vehicle damage,* and *third-party liability* requires domain knowledge that a basic CFG does not provide.

c) Improved parsing approach using PCFG, Feature Structures and Earley Parsing A more robust architecture can combine three techniques:

Customer Statement

↓

Text Preprocessing

↓

Tokenization

↓

Earley Parser

↓

CFG Candidate Parse Trees

↓

PCFG Probability Ranking

↓

Feature Structure Validation

↓

Semantic Role Extraction

↓

Insurance Claim Representation

PCFG

A Probabilistic Context-Free Grammar assigns probabilities to grammar rules.

For example:

VP → V NP PP 0.65

VP → V PP NP 0.20

NP → NP PP 0.15

If historical insurance data shows that **"damage to the car"** is much more common than the alternative interpretation, the corresponding parse can receive a higher probability.

Therefore, PCFG helps select the **most likely interpretation** instead of treating every parse equally.

Feature Structures

Feature structures can represent grammatical information:

NP:

number = singular

person = third role

= claimant For

example: Customer + reports is compatible because the subject and verb agree.

Feature structures can also represent semantic information:

Object:

type = vehicle

object = car

This helps distinguish an object of damage from a destination or recipient.

Earley Parsing

Earley Parsing is suitable for natural-language input because it can efficiently handle CFG grammars and ambiguity while processing input from left to right.

It can maintain multiple possible partial parses rather than repeatedly restarting the entire parsing process.

This is particularly useful when customer statements are long or incomplete.

d) Benefits of the proposed architecture

The proposed architecture improves the insurance system in several ways.

Accuracy: PCFG selects the most probable interpretation while feature structures enforce grammatical and semantic constraints.

Scalability: Earley Parsing can process different sentence structures without requiring a separate parser for every sentence pattern.

Reliability: Agreement checking and semantic validation reduce incorrect extraction of claim information.

Better claim extraction: The system can identify:

Claimant → Customer

Damaged Object → Car

Cause → Accident

Action → Reported

Claim Amount → Extracted from claim statement

Conversational support: Partial and informal customer statements can be processed more effectively. Therefore, integrating **CFG + PCFG + Feature Structures + Earley Parsing** provides a stronger foundation for automated insurance claim processing.

2. Intelligent Railway Ticket Booking Assistant

a) CFG analysis of the passenger request

The sentence is:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper." A simplified CFG can be:

S → VP

VP → V NP

NP → Det Num N PP PP PP

PP → P NP

One possible interpretation is:

[Book [two tickets]
[from Chennai]
[to Delhi]
[for my parents]
[with AC sleeper]]

Here:

- **two tickets** → quantity **from Chennai** → source
- **to Delhi** → destination
- **for my parents** → passengers
- **with AC sleeper** → ticket/class information

However, another structural interpretation can attach **"with AC sleeper"** to the passenger phrase: [for [my parents [with AC sleeper]]]

This could incorrectly suggest that *AC sleeper* describes the parents rather than the ticket class.

The ambiguity is mainly caused by **prepositional phrase attachment**.

b) Problems with top-down parsing

Top-down parsing starts from the start symbol and attempts to construct a parse tree toward the input.

For example:

S

↓

VP

↓

NP + PP

↓

...

The major problem is **backtracking**. Suppose

the parser initially assumes:

with AC sleeper → passengers

After discovering that this interpretation is incorrect, it must return to an earlier decision and try another possibility.

For long conversational requests, many alternative structures may be generated.

This causes:

- excessive backtracking
- increased processing time
- repeated grammar-rule evaluation
- poor performance for ambiguous sentences
- difficulty handling incomplete requests

For real-time railway booking, delays are undesirable because passengers expect immediate responses.

c) Earley Parsing for the railway assistant

Earley Parsing processes the sentence incrementally and maintains possible states.

For example:

Book two tickets from Chennai to Delhi

The parser can maintain the incomplete structure while waiting for additional information.

When:

for my parents is received, it can extend the passenger-

related structure.

When:

with AC sleeper is received, multiple interpretations can
initially be maintained.
PCFG or semantic constraints can then prefer:
with AC sleeper → ticket class rather
than:
with AC sleeper → passengers

Earley Parsing is therefore useful for:

- ambiguous input
- partial input
- incremental processing
- different sentence structures
- conversational queries

d) Comparison

Feature	**Top-Down Parsing**	**CFG-Based Parsing**	**Earley Parsing**
Basic approach	Starts from start symbol	Uses grammar rules	Dynamic ogrammin g with chart
Ambiguity	Poor handling	Can generate multiple parses	Maintains multiple possibilities
Backtracking	High	Depends on implementation	Greatly reduced
Partial input	Limited	Limited	Strong
Long sentences	Can become inefficient	Can generate many structures	More suitable
Complexity	Can be high with backtracking	Depends on parser	$O(n^3)$ worst case for general CFG
Real-time suitability	Low for complex queries	Moderate	High
Conversational input	Weak	Moderate	Strong

For a real-time railway booking assistant, **Earley Parsing combined with probabilistic and semantic constraints** is more suitable than a conventional top-down parser.

3. AI-Based Legal Contract Analysis System

a) CFG analysis of the contract sentence The sentence is:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

A simplified structure is:

S

├── NP

│ ├── Det → The

│ ├── N → supplier

│ └── Relative Clause

│ ├── who

│ ├── delivered

│ ├── NP → the equipment

│ └── PP → to the company

└── VP

├── must

├── replace

├── NP → the defective components

└── PP → within thirty days The

intended interpretation is:

Supplier

↓

delivered equipment

↓

to company

Supplier

↓

must replace defective components

↓

within thirty days

A possible source of ambiguity is the PP:

to the company

b) Why basic CFG is insufficient

Basic CFG has difficulty with complex legal sentences because legal language frequently contains:

- relative clauses
- passive constructions
- nested clauses
- multiple prepositional phrases
- long-distance dependencies
- coordination
- exceptions
- conditional obligations

A CFG can identify syntactic structures but does not inherently understand the semantic relationship between entities.

For example, it may identify:

supplier equipment company without reliably
determining: supplier → delivered → equipment
supplier → delivered equipment → to company
supplier → must replace → defective components
deadline → thirty days

Legal documents require both **syntactic analysis and semantic interpretation**.

c) Proposed NLP architecture The
proposed architecture is:
Legal Contract
↓
Document Preprocessing
↓
Sentence Segmentation
↓
Tokenization
↓
CFG Grammar

↓

Earley Parsing

↓

Multiple Candidate Parse Trees

↓

PCFG Ranking

↓

Feature Structure Validation

↓

Semantic Role / Entity Extraction

↓

Obligation & Deadline Detection

↓

Structured Contract Representation

CFG

CFG provides the basic syntactic grammar.

Earley Parsing

Earley Parsing handles complex and nested structures while maintaining possible interpretations.

PCFG

PCFG assigns probabilities to candidate structures.

The most probable legal interpretation can therefore be selected.

Feature Structures

Feature structures can represent:

Entity: type =

Organization role =

Supplier and:

Obligation:

subject = Supplier action =

Replace object = Defective

Components deadline = 30 days

Semantic constraints can then verify whether the extracted relationships are consistent.

d) Step-by-step contract processing workflow Step 1 – Input

The supplier who delivered the equipment to the company must replace the defective components within thirty days.

Step 2 – Tokenization

The sentence is divided into tokens:

The | supplier | who | delivered | the | equipment | to | the | company | must | replace | the | defective | components | within | thirty | days

Step 3 – CFG parsing The system identifies:

Main Subject → supplier

Relative Clause → who delivered the equipment to the company

Main Action → must replace

Object → defective components

Deadline → within thirty days

Step 4 – Earley Parsing

Earley Parsing maintains candidate structures and handles the relative clause and PP attachments. **Step 5 – PCFG ranking**

The candidate interpretation that gives: supplier → delivered → equipment is preferred over an incorrect interpretation involving the company as the delivery agent.

Step 6 – Feature validation

The system checks semantic roles:

Supplier = Agent

Equipment = Object

Company = Destination

Step 7 – Information extraction

The final structured representation becomes:

Information	**Extracted value**
Supplier	The supplier
Action	Delivered / Replace
Equipment	The equipment
Company	The company
Obligation	Replace defective components
Deadline	Within thirty days

Thus, the system understands not only the grammar but also the **relationship between legal entities and obligations**.

4. AI-Based E-Commerce Product Search System

a) CFG analysis of the query The

query is:

"Find laptops with a touchscreen for students under ₹60,000."

A simplified interpretation is: Find

└── laptops

  ├── with a touchscreen

  ├── for students

  └── under ₹60,000

The intended semantic representation is:

Product = Laptop

Feature = Touchscreen

Target User = Students

Maximum Price = ₹60,000 However,

the phrase:

for students can

potentially attach to:

touchscreen

or:

laptops

The intended interpretation is:

[laptops [with a touchscreen] [for students] [under ₹60,000]] rather

than:

[laptops [with [a touchscreen for students]] [under ₹60,000]]

The second interpretation could incorrectly treat *for students* as a property of the touchscreen.

b) Limitations of CFG

Basic CFG faces several difficulties in conversational e-commerce queries.

Ambiguous attachment

It may not determine which noun a PP modifies.

Missing words Users

may type:

laptops touchscreen students 60k instead

of a complete sentence.

Informal language Examples

include:

Need laptop below 60k for college. or:

Best touch laptop under 50k.

Ellipsis

Users frequently omit grammatical components:

Laptop under 60k, touchscreen, student use.

Long queries

A customer may specify many attributes:

Find a lightweight laptop with touchscreen,

16GB RAM, 512GB SSD, good battery life,

for engineering students under ₹70,000.

A basic CFG may generate many possible structures.

Real-time requirement

An e-commerce search system must interpret queries rapidly and return products immediately.

c) Improved architecture

The proposed architecture is:

Customer Query

↓

Normalization

↓

Tokenization

↓

Earley Parsing

↓

CFG Candidate Structures

↓

PCFG Probability Ranking

↓

Feature Structure Validation

↓

Semantic Attribute Extraction

↓

Search Query Generation

↓

Product Database/Search Engine

↓

Ranked Products

PCFG

PCFG can rank alternative interpretations. For example:

Laptop + for students → High probability

Touchscreen + for students → Lower probability The system therefore selects the intended interpretation.

Feature Structures

The extracted query can be represented as:

PRODUCT:

category = laptop

touchscreen = yes target_user = students max_price = 60000

d) Benefits Improved search precision

Semantic features prevent incorrect interpretation of attributes.

Faster response

Efficient parsing reduces unnecessary parsing and ambiguity-handling overhead.

Better customer experience

Customers can use natural conversational language rather than rigid filters.

For example:

Need a laptop for college under 60k with touchscreen. can be converted into:

Category = Laptop

User = Student

Touchscreen = Yes

Price ≤ ₹60,000

The search engine can then return appropriate products.

Therefore, combining **PCFG + Feature Structures + Earley Parsing** improves natural-language product search in large e-commerce systems.

5. Intelligent Airline Voice Assistant

a) Syntactic structures and ambiguities The spoken command is:

"Change my flight to Mumbai tomorrow with two passengers."

A possible structure is: Change

└── my flight

├── to Mumbai

├── tomorrow

└── with two passengers The intended interpretation is:

Action → Change

Booking → My flight

Destination → Mumbai

Date → Tomorrow

Passenger Count → 2

However, a parser may incorrectly associate:

with two passengers

with the destination phrase:

Mumbai with two

passengers instead of the

booking. The second

example is:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore." This contains a **correction**.

The initial destination is:

Mumbai

but the user later changes it to:

Bangalore

The final interpretation should therefore be:

Destination = Bangalore rather than

Mumbai.

b) Limitations of top-down parsing

Top-down parsing is problematic for voice assistants because speech input is often:

- incomplete

- noisy
- ambiguous
- corrected
- interrupted
- grammatically irregular For example:

Change my flight to Mumbai...

The parser may begin constructing a complete structure before the user finishes speaking.

Then:

actually, make that Bangalore requires the system to

revise the previous interpretation.

A conventional top-down parser may need to backtrack and rebuild the parse.

This produces:

- higher latency
- repeated parsing
- poor handling of corrections
- difficulty with partial input
- inefficient ambiguity resolution

For real-time airline systems, these delays can negatively affect the user experience.

c) Proposed architecture The

proposed architecture is:

Passenger Speech

↓

Speech Recognition

↓

Text Normalization

↓

Incremental Input

↓

Earley Parser

↓

Candidate Parse Structures

↓

PCFG Ranking

↓

Feature Structure Validation

↓

Correction / Context Resolution

↓

Semantic Intent Extraction

↓

Structured Booking Request

↓

Airline Booking System

Earley Parsing

Earley Parsing is useful for incremental and partial input.

The parser can maintain:

Change my flight to Mumbai as a partial parse while

waiting for additional information.

PCFG

PCFG can rank possible interpretations based on learned probabilities.

For example: with two passengers → passenger count can receive

a higher probability than attaching it to the destination.

Feature Structures

The system can represent the booking request as:

BOOKING:

action = change

destination = Mumbai date =

tomorrow passengers = 2

After the correction:

destination = Bangalore the

feature structure is updated.

The final request becomes:

BOOKING:

action = change

destination = Bangalore

date = tomorrow

passengers = 2

d) Processing workflow Step 1 – Speech recognition

The voice assistant converts speech to text:

Change my flight to Mumbai tomorrow with two passengers.

Step 2 – Incremental parsing

Earley Parsing processes the command while maintaining partial structures.

Step 3 – Candidate generation

Possible interpretations are generated:

Destination = Mumbai Passenger

Count = 2

and alternative attachment structures.

Step 4 – PCFG ranking

PCFG assigns probabilities to candidate parses.

The interpretation where **"with two passengers"** represents passenger count receives a higher probability.

Step 5 – Feature validation Feature structures verify:

destination → city date → valid

temporal expression passengers

→ numeric value

Step 6 – Correction detection The

system receives:

Actually, make that Bangalore.

The correction mechanism identifies that **Bangalore replaces Mumbai**.

Step 7 – Final structured request

Action: Change Flight

Destination: Bangalore

Date: Tomorrow

Passengers: 2

Step 8 – Airline system execution

The structured request is passed to the airline booking system, which searches for available flights matching the updated requirements.